TowerWhisper — Tech Spec

Craiova Hackathon v2 · 30 octombrie 2026 · Ramon Agud · v2 (Comms Watchdog)

TowerWhisper e o aplicație web care ascultă controlorul de trafic, înțelege instrucțiunile și alertează când pilotul confirmă greșit sau avionul nu execută ce i s-a cerut. Arată și de cât timp nu a mai comunicat fiecare avion: galben la 15 minute, roșu la 30.

1. Problema și pitch-ul

Pitch într-o frază: **radarul arată controlorului unde sunt avioanele; TowerWhisper îi spune când un avion nu a înțeles sau nu face ce i s-a spus.**

- **Vocea pe VHF e încă canalul principal** în apropiere și în turn. Datalink-ul text (CPDLC) e folosit mai ales în spațiul aerian superior.
- **Readback/hearback:** pilotul repetă instrucțiunea greșit, iar controlorul, ocupat cu alte avioane, nu observă. E un factor cunoscut în level bust-uri și incursiuni pe pistă.
- **Nimic nu leagă azi vocea de radar.** Sistemele știu unde e avionul, dar nu știu ce i s-a spus.

Deschiderea pitch-ului (fără slide-uri, direct în aplicație): „Un controlor dă zeci de instrucțiuni pe oră și trebuie să asculte fiecare confirmare. Dacă pilotul spune 5000 în loc de 3000 și controlorul e ocupat, nu prinde nimeni. Puteți încerca chiar voi.” Apoi un membru al juriului ia microfonul.

Cifre: folosiți doar ce măsurați voi (acuratețea parser-ului, latența) sau o cifră cu sursă dintr-un raport EUROCONTROL/EASA. Fără statistici fără sursă.

2. Scope

MVP-ul e tot ce trebuie pentru demo-ul complet: userul vorbește, pilotul simulat răspunde, erorile se activează din panou și alertele apar.

MVP (obligatoriu):

- Simulator de trafic în jurul Craiovei (LRCV, pista 09/27) cu 6–8 avioane
- Push-to-talk din browser → transcriere → parser → instrucțiune structurată
- 5 tipuri de instrucțiuni: altitudine/nivel, cap, viteză, aprobare de apropiere ILS, schimbare de frecvență
- Pilot simulat care confirmă vocal (TTS cu efect radio) și execută manevra
- Readback Check, Conformance Monitor și Comms Watchdog (timp de la ultima comunicare)
- Panoul de demo cu erori activabile (secțiunea 11)
- Ecran radar cu etichete, transcriere live, fișe de zbor, alerte cu sunet

Bonus, în ordinea valorii: Clearance Probe → jurnal al sesiunii cu replay → fundal cu trafic real OpenSky → răspunsuri libere ale pilotului generate de LLM.

Ce nu facem: integrare cu sisteme ATC reale, recunoaștere de vorbitor, alte limbi decât engleza ATC, autentificare de utilizatori, aplicație mobilă.

Ce se taie dacă rămâneți în urmă: frecvența și viteza din parser, apoi vocea pilotului (rămâne text pe ecran). Readback Check, Conformance, Comms Watchdog și panoul de demo nu se taie niciodată.

3. Fluxul utilizatorului

Userul e controlorul: vorbește în microfon exact ca pe frecvență, iar restul se întâmplă automat.

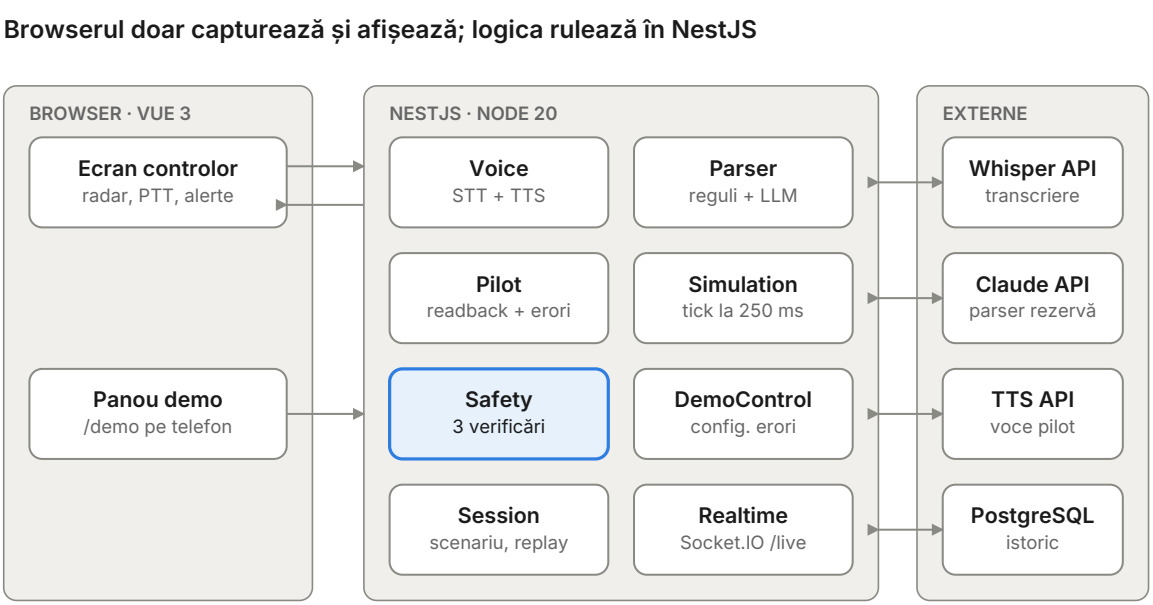
1. Userul selectează un avion pe radar (opțional; sistemul identifică și singur callsign-ul din frază).
2. Ține apăsat **Space** sau butonul PTT și spune: *„TAROM 301, descend altitude 3000 feet, turn left heading 120”*. Un VU-metru arată că microfonul prinde sunetul.
3. La eliberarea tastei, audio-ul pleacă la backend. În ~1–2 secunde apare transcrierea în panoul de comunicații, cu cuvintele-cheie evidențiate.
4. Instrucțiunea parsată apare ca „chip-uri” (`ALT 3000 ↓`, `HDG 120 L`), iar avionul vizat se aprinde pe radar. Fișa lui de zbor se actualizează.
5. După ~1,5 secunde, pilotul simulat răspunde vocal, cu efect radio: *„Descend 3000, left heading 120, TAROM 301”*.
6. Readback Check compară răspunsul cu instrucțiunea: bifă verde sau alertă roșie `HEARBACK MISMATCH`.

7. Avionul începe manevra pe radar. Conformance Monitor urmărește execuția până la atingerea țintei.
8. La o alertă, userul o confirmă (Acknowledge) sau corectează vorbind din nou: *„TAROM 301, negative, descend 3000”*.

Userul nu folosește niciodată tastatura pentru instrucțiuni. Tastatura e doar pentru PTT și scurtăturile panoului de demo.

În fundal, Comms Watchdog urmărește fiecare avion. Orice transmisie a controlorului către el sau a pilotului resetează un timer, afișat ca `LC 08:35` pe radar și `LAST CALL` pe fișa de zbor. La 15 minute fără comunicare avionul devine galben, la 30 minute roșu, cu alertă `NO_CONTACT`.

4. Arhitectura



Arhitectura TowerWhisper

Browserul trimite audio-ul la eliberarea PTT și primește totul înapoi pe socket. Toate deciziile de siguranță se iau în modulul Safety, fără apeluri externe; API-urile de AI sunt folosite doar pentru transcriere, voce și frazele pe care regulile nu le recunosc.

5. Tech stack

Tot stack-ul e TypeScript, web-only, pe tehnologii pe care le știți deja: Vue 3 în față, NestJS în spate, PostgreSQL pentru persistență. Singurele piese externe sunt două API-uri de AI.

Strat	Tehnologie	Rol
Frontend	Vue 3 + Vite + TypeScript	SPA, fără SSR (Nuxt nu aduce nimic aici)
Stare frontend	Pinia	Store-uri pentru trafic, comunicații, alerte, demo
Radar	Canvas 2D nativ	Desenat la 60 fps, fără bibliotecă de hărți
Audio în browser	MediaRecorder + Web Audio API	Înregistrare PTT, VU-metru, filtru radio pentru pilot
Timp real	Socket.IO (client + `@nestjs/websockets`)	Starea avioanelor la 4 Hz, evenimente de comunicații și alerte
Backend	Node.js 20 + NestJS + TypeScript	Simulator, parser, motor de siguranță, API
Bază de date	PostgreSQL 16 + TypeORM	Sesiuni, transmisii, instrucțiuni, alerte (pentru replay și statistici)
Speech-to-text	OpenAI Whisper API (`whisper-1`)	Transcriere; parametrul `prompt` primește callsign-urile și vocabularul ATC
LLM de rezervă	Claude API (Haiku), output JSON prin tool use	Doar când parser-ul pe reguli nu recunoaște fraza

Text-to-speech	OpenAI TTS API (MP3)	Vocea pilotului, trecută prin filtrul radio
Dev	Docker Compose, monorepo pnpm	Postgres local + pachet comun `shared` cu tipurile

Atenție la TTS: vocea nativă a browserului (`speechSynthesis`) nu poate fi trecută prin Web Audio, deci nu puteți aplica filtrul radio pe ea. Folosiți un API de TTS care returnează fișier audio. Păstrați `speechSynthesis` doar ca rezervă offline, cu zgomot radio redat în paralel.

Structura repo-ului:

```
towerwhisper/
  apps/web/        Vue 3 + Vite
  apps/api/        NestJS
  packages/shared/ tipuri TS comune (Aircraft, Clearance, Alert, evenimente WS)
  docker-compose.yml
```

6. Backend: module NestJS

Backend-ul e sursa unică de adevăr: simulatorul, pilotul și motorul de siguranță rulează toate în NestJS, iar frontend-ul doar afișează și capturează audio.

Modul	Responsabilitate	Clase principale
`SessionModule`	Pornește, resetează și salvează o sesiune de demo cu scenariul ales	`SessionService`, `ScenarioLoader`
`SimulationModule`	Bucla de tick la 250 ms: actualizează poziția, capul, altitudinea, viteza fiecărui avion spre țintele lui	`SimulationService`, `AircraftKinematics`
`VoiceModule`	Primește audio de la PTT, apelează STT; generează audio TTS pentru pilot	`SttService`, `TtsService`
`ParserModule`	Text → `Clearance`: normalizare, reguli, LLM de rezervă, potrivire callsign	`PhraseNormalizer`, `RuleParser`, `LlmParser`, `CallsignResolver`
`PilotModule`	Construiește readback-ul, aplică erorile din panoul de demo, programează răspunsul și execuția	`PilotService`, `ReadbackBuilder`
`SafetyModule`	Readback Check, Conformance Monitor la fiecare tick, Clearance Probe	`ReadbackChecker`, `ConformanceMonitor`, `ClearanceProbe`
`DemoControlModule`	Ține configurația de erori și declanșează scenarii (conflict, zgomot, transmisie blocată)	`FaultService`
`RealtimeModule`	Gateway Socket.IO: trimite evenimentele spre toate ecranele conectate	`RealtimeGateway`

Fluxul unei transmisii, pas cu pas:

1. `POST /api/transmissions` cu audio (webm/opus) → `SttService` → text.
2. `ParserModule` → `Clearance` + salvare `Transmission` și `Clearance` în Postgres → evenimentele `comms:transmission` și `clearance:issued`.
3. `PilotService` citește erorile active, construiește readback-ul (corect sau alterat) și îl generează audio cu `TtsService`.
4. Readback-ul trece tot prin `ParserModule`, apoi prin `ReadbackChecker` → `comms:readback` + eventual `alert:raised`.
5. `PilotService` aplică țintele în simulator (corecte sau alterate de eroarea de execuție), după o întârziere de reacție de 2–3 secunde.
6. `ConformanceMonitor` verifică la fiecare tick și ridică alerta când deviația persistă.

Readback-ul pilotului se generează din șablon, deci e determinist. Asta face demo-ul repetabil.

7. Modelul de date

Tipurile stau în `packages/shared` și sunt folosite identic în Vue și NestJS. Starea live a avioanelor stă în memorie; în Postgres se salvează doar ce e nevoie pentru replay și statistici.

```
        // packages/shared/src/types.ts
```

```ts
export type TurnDir = 'left' | 'right' | 'any';

export interface Aircraft {
  callsign: string;        // 'ROT301'
  telephony: string;       // 'TAROM 301'
  type: string;            // 'AT76'
  lat: number; lon: number;
  altitudeFt: number;
  headingDeg: number;      // 0-359
  speedKt: number;
  verticalSpeedFpm: number;
  turnRateDegS: number;    // + dreapta, - stânga
  target: Targets;         // spre ce converge simulatorul
  activeClearanceId?: string;
  lastContactAt: string;   // ultima transmisie ATC sau pilot cu acest avion (timp simulat)
  commsAgeS: number;       // calculat de backend la fiecare tick
}

export interface Targets {
  altitudeFt?: number;
  headingDeg?: number;
  turnDir?: TurnDir;
  speedKt?: number;
  approach?: { type: 'ILS'; runway: '09' | '27' };
}

export interface Clearance {
  id: string;
  callsign: string;
  speaker: 'atc' | 'pilot';
  altitude?: { valueFt: number; action: 'climb' | 'descend' | 'maintain'; flightLevel: boolean };
  heading?: { valueDeg: number; turn: TurnDir };
  speed?: { valueKt: number };
  approach?: { type: 'ILS'; runway: string };
  frequency?: { mhz: string };
  rawText: string;
  confidence: number;      // 0-1
  parsePath: 'rules' | 'llm';
}

export type AlertType = 'HEARBACK_MISMATCH' | 'READBACK_INCOMPLETE' | 'WRONG_TURN'
  | 'LEVEL_BUST' | 'WRONG_VERTICAL' | 'NO_RESPONSE' | 'NO_CONTACT' | 'CONFLICT_PREDICTED';

export interface Alert {
  id: string;
  type: AlertType;
  severity: 'caution' | 'warning';  // galben / roșu
  callsigns: string[];
  message: string;         // 'ROT301 autorizat 3000, readback 5000'
  details: Record<string, unknown>;
  raisedAt: string;
  acknowledgedAt?: string;
}
```

Tabele PostgreSQL (TypeORM, cu `uuid` ca cheie):

Tabel	Coloane principale
`sessions`	id, scenario, started_at, ended_at
`transmissions`	id, session_id, speaker, audio_path, transcript, stt_latency_ms, created_at
`clearances`	id, transmission_id, callsign, payload (jsonb), parse_path, confidence
`alerts`	id, session_id, type, severity, callsigns (text[]), message, details (jsonb), raised_at, acknowledged_at
`fault_events`	id, session_id, fault_type, callsign, config (jsonb), triggered_at

`fault_events` vă permite să arătați la final: „am injectat 6 erori, sistemul le-a prins pe toate, timp mediu de detecție X secunde".

8. Contractul REST și WebSocket

REST e doar pentru comenzi; tot ce se schimbă în timp real vine pe Socket.IO, namespace-ul `/live`.

REST (`/api`):

Metodă	Ruta	Body / răspuns
POST	`/sessions`	`{ scenario: 'craiova-arrivals' }` → `{ sessionId }`
POST	`/sessions/:id/reset`	repornește scenariul de la zero
POST	`/transmissions`	multipart: `audio` (webm), `sessionId`, `selectedCallsign?` → `{ transmissionId, transcript, clearance }`

POST	`/transmissions/text`	`{ text }`, pentru teste și ca rezervă dacă pică microfonul
GET	`/demo/faults`	configurația curentă de erori
PUT	`/demo/faults`	`FaultConfig` (secțiunea 11)
POST	`/demo/triggers/:name`	`conflict`, `stepped-on`, `no-response`
POST	`/alerts/:id/ack`	marchează alerta ca văzută
GET	`/sessions/:id/stats`	erori injectate, detectate, timp mediu de detecție, latență STT

Pentru Comms Watchdog: `POST /demo/triggers/age-comms` cu `{ callsign, ageS }` îmbătrânește timerul unui avion, iar `POST /demo/triggers/reset-comms` cu `{ callsign }` îl resetează. Limitele se schimbă prin `PUT /demo/faults` (câmpul `commsLimits`). `traffic:update` include deja `lastContactAt` și `commsAgeS` pentru fiecare avion.

Evenimente server → client:

Eveniment	Payload	Frecvență
`traffic:update`	`Aircraft[]`	la fiecare tick, 4 Hz
`comms:transmission`	`{ id, speaker: 'atc', transcript, clearance }`	după fiecare PTT
`comms:readback`	`{ id, speaker: 'pilot', callsign, transcript, clearance, audioUrl, check }`	după fiecare readback
`clearance:issued`	`Clearance`	odată cu transmisia
`alert:raised`	`Alert`	la detecție
`alert:cleared`	`{ id, reason }`	când condiția dispare
`probe:result`	`{ callsign, path: LatLon[], conflicts: [...] }`	bonus, Clearance Probe
`demo:faults`	`FaultConfig`	la orice schimbare din panou

`check` în `comms:readback` ia valorile `'ok'`, `'mismatch'` sau `'incomplete'`.

Client → server: `ptt:start` și `ptt:end` (doar ca să arătați pe toate ecranele că frecvența e ocupată). Audio-ul pleacă prin REST, nu prin socket.

9. Parser-ul de frazeologie ATC

Parser-ul e determinist pentru frazele standard; LLM-ul intervine doar când regulile nu găsesc nimic. Același parser procesează și instrucțiunea controlorului, și readback-ul pilotului.

Pasul 1, prompt pentru STT. Parametrul `prompt` al Whisper primește o frază cu vocabularul sesiunii, ca transcrierea să nimerească termenii: *„TAROM 301, WIZZ 44, BLUE AIR 12, descend, climb, flight level, heading, cleared ILS approach runway zero niner."*

Pasul 2, normalizare (`PhraseNormalizer`): litere mici, punctuație scoasă, apoi înlocuiri:

Intrare	Ieșire
`niner`, `nine`	`9`
`tree`, `three`	`3`
`fife`, `five`	`5`
`one two zero`	`120`
`three thousand`, `3,000`	`3000`
`flight level six zero`, `FL 60`	`FL060`
`runway zero niner`	`RWY09`

Pasul 3, reguli (`RuleParser`): fiecare tip de instrucțiune e un regex peste textul normalizat. Fraza poate conține mai multe instrucțiuni; se aplică toate regulile.

```
const RULES = {
  altitude: /\b(climb|descend|maintain)\b(?: and maintain)?(?: to)?\s+(?:(FL)(\d{2,3})|altitude\s+(\d{3,5})|(\d{3,5})\s*(?:feet|ft)?)/,
  heading:  /\bturn\s+(left|right)\s+heading\s+(\d{3})\b|\bfly\s+heading\s+(\d{3})\b/,
  speed:    /\b(?:reduce|increase|maintain)?\s*speed\s+(\d{3})\b/,
  approach: /\bcleared\s+ils\s+(?:approach\s+)?RWY(\d{2})\b/,
  frequency:/\bcontact\s+\w+\s+(\d{3})\s*(?:decimal|point)?\s*(\d{1,3})\b/,
};
```

Pasul 4, callsign (`CallsignResolver`): se compară începutul frazei (controlor) sau sfârșitul ei (pilot) cu telefoniile avioanelor active. Potrivire fuzzy (distanță Levenshtein ≤ 2 pe nume, cifre exacte). Dacă userul a selectat un avion pe radar și callsign-ul nu e clar, se folosește selecția. Fără callsign valid → alertă galbenă „Callsign neidentificat".

Pasul 5, LLM de rezervă (`LlmParser`), apelat doar dacă nicio regulă nu a prins: Claude cu un singur tool `emit_clearance` al cărui schema JSON e exact tipul `Clearance`, plus lista avioanelor active în prompt. Timeout 2 secunde; dacă expiră, transmisia e marcată „neparsată".

Setul de test: un fișier `phrases.json` cu 60–100 de fraze (text așteptat + `Clearance` așteptat). Un test Jest raportează acuratețea per câmp. Pe aceeași listă înregistrați și audio cu vocile voastre, pentru cifra din pitch.

10. Motorul de siguranță

Toate verificările sunt calcul determinist în TypeScript, fără AI. Fiecare alertă conține motivul în clar, ca controlorul să o verifice dintr-o privire.

Readback Check (`ReadbackChecker`), rulat o dată la fiecare readback:

Situație	Alertă	Severitate
Un câmp din readback are altă valoare (altitudine, cap, pistă, frecvență)	`HEARBACK_MISMATCH`	roșu
Sensul virajului e altul (left vs right)	`HEARBACK_MISMATCH`	roșu
Readback-ul a fost făcut de alt callsign decât cel instruit	`HEARBACK_MISMATCH`	roșu
Lipsește un câmp din readback	`READBACK_INCOMPLETE`	galben
Niciun readback în 8 secunde	`NO_RESPONSE`	galben

Conformance Monitor (`ConformanceMonitor`), rulat la fiecare tick pentru avioanele cu instrucțiune activă. O alertă se ridică doar dacă deviația persistă **4 secunde** (evită alarmele false în timpul tranziției):

- `WRONG_TURN`: instrucțiunea cere `left`, dar `turnRateDegS > +0,5` (sau invers).
- `WRONG_VERTICAL`: avionul urcă când trebuie să coboare (sau invers), cu `|verticalSpeedFpm| > 300`.
- `LEVEL_BUST`: avionul e la mai puțin de 300 ft de țintă, dar viteza verticală nu scade sub 1000 ft/min. Timpul până la depășire se estimează și apare în mesaj: *„ROT301 va depăși 3000 ft în 12 s"*.
- Alerta dispare automat (`alert:cleared`) când avionul revine în conformitate.

Comms Watchdog (`CommsWatchdog`), rulat la fiecare tick pentru fiecare avion:

- `lastContactAt` se actualizează la orice transmisie a controlorului către avion și la orice transmisie a pilotului, readback inclus. Selecția pe radar sau ACK-ul unei alerte nu resetează timerul.
- `commsAgeS` = timpul simulat curent − `lastContactAt`, deci respectă viteza timpului din panoul de demo.
- Pragurile vin din env (`COMMS_CAUTION_MIN=15`, `COMMS_WARNING_MIN=30`) și pot fi schimbate din panoul de demo.

Prag	Ce se vede	Alertă
sub 15 min	`LC mm:ss` gri pe radar și pe fișă	—
≥ 15 min	LC galben, avionul galben pe radar, marginea fișei galbenă	`NO_CONTACT`, galben
≥ 30 min	LC roșu, avionul roșu, marginea fișei roșie	`NO_CONTACT`, roșu (înlocuiește alerta galbenă)

Mesajul alertei: *„No communication with DLH9TC for 31 min (limit 30 min)"*. Alerta dispare automat (`alert:cleared`) la următoarea transmisie cu acel avion.

Clearance Probe (`ClearanceProbe`, bonus): la fiecare instrucțiune nouă, proiectează traiectoria rezultată la pas de 5 secunde, pe 5 minute, și calculează apropierea minimă față de celelalte avioane. Cu p = poziția relativă și v = viteza relativă:

$$t^* = -(p \cdot v) / \|v\|^2, \quad d_{min} = \|p + v \cdot t^*\|$$

Alertă `CONFLICT_PREDICTED` dacă t* e între 0 și 5 minute, apropierea orizontală e sub 3 NM și diferența de altitudine sub 1000 ft. Pentru virajele din traiectorie, simularea pas cu pas e mai simplă decât formula; formula rămâne pentru segmentele drepte și pentru slide.

Simulatorul folosește valori realiste, ca manevrele să arate credibil pe radar: viraj standard de 3°/s, urcare/coborâre 1500–2000 ft/min, schimbare de viteză 2 kt/s, conversie la lat/lon cu aproximare plană locală (suficientă pe 50 km).

11. Panoul de demo

Panoul de demo e o rută separată, `/demo`, pe care o deschideți pe un telefon sau pe al doilea laptop. Ecranul mare rămâne curat, iar juriul nu vede când apăsați butonul. Ambele ecrane sunt sincronizate prin Socket.IO. Panoul există și ca sertar lateral pe ecranul principal, deschis cu tasta **D**.

Cum se aplică o eroare: alegeți ținta (avionul selectat / un callsign anume / oricare) și modul: **o dată** (doar următoarea transmisie, apoi se dezactivează singură) sau **permanent**. Modul „o dată" e cel folosit în demo.

Erori de readback (pilotul confirmă greșit):

Eroare	Ce spune pilotul	Alertă așteptată
Altitudine greșită	5000 în loc de 3000 (decalaj configurabil)	`HEARBACK_MISMATCH`
Cap greșit	cifre inversate: 210 în loc de 120	`HEARBACK_MISMATCH`
Sens de viraj greșit	„right heading 120"	`HEARBACK_MISMATCH`
Pistă greșită	„runway 27" în loc de 09	`HEARBACK_MISMATCH`
Callsign greșit	alt avion („WIZZ 44") confirmă instrucțiunea pentru ROT301	`HEARBACK_MISMATCH`
Readback incomplet	omite altitudinea	`READBACK_INCOMPLETE`
Fără răspuns	tăcere	`NO_RESPONSE` după 8 s

Erori de execuție (pilotul confirmă corect, dar avionul face altceva):

Eroare	Ce face avionul	Alertă așteptată
Viraj în sens opus	virează la dreapta	`WRONG_TURN`
Depășire de nivel	coboară până la 1500 în loc de 3000	`LEVEL_BUST`
Sens vertical opus	urcă în loc să coboare	`WRONG_VERTICAL`
Ignoră instrucțiunea	își păstrează capul și altitudinea	`WRONG_TURN` / `WRONG_VERTICAL`

Comms timer (testează Comms Watchdog):

Control	Efect	Alertă așteptată
Limite 15 / 30 min ↔ 15 / 30 s	modul demo scurtează pragurile, ca juriul să vadă alerta apărând live	—
Age to 15 min	timerul țintei sare la pragul galben	`NO_CONTACT` galben
Age to 30 min	timerul țintei sare la pragul roșu	`NO_CONTACT` roșu
Reset timer on target	`lastContactAt` = acum	alerta dispare

În demo: apăsați Age to 30 min pe un avion, apoi controlorul îl cheamă la microfon. Alerta dispare imediat ce transmisia ajunge.

Scenarii și control:

- **Conflict:** aduce un al doilea avion pe un curs convergent, ca să arătați Clearance Probe.
- **Transmisie blocată (stepped-on):** suprapune zgomot peste readback, care ajunge parțial; sistemul cere repetare.
- **Nivel de zgomot radio:** glisor 0–100% pe vocea pilotului și opțional peste microfonul userului înainte de STT, ca să arătați robustețea.
- **Timp:** pauză, 1×, 2×, 4× (avioanele ajung mai repede la țintă în demo).
- **Reset scenariu** și **contor live**: erori injectate / detectate / timp mediu de detecție.
- **Scurtături:** tastele 1–9 activează erorile în modul „o dată", R resetează scenariul.

```ts
export interface FaultConfig {
  target: { mode: 'selected' | 'callsign' | 'any'; callsign?: string };
  readback: null | {
    kind: 'wrong_altitude' | 'wrong_heading' | 'wrong_turn' | 'wrong_runway'
        | 'wrong_callsign' | 'incomplete' | 'no_response';
    altitudeOffsetFt?: number;
    once: boolean;
  };
  execution: null | {
    kind: 'turn_wrong_way' | 'level_bust' | 'wrong_vertical' | 'ignore';
    overshootFt?: number;
    once: boolean;
  };
  radioNoise: number;       // 0-1
  timeScale: 0 | 1 | 2 | 4;
  commsLimits: { cautionS: number; warningS: number };  // implicit 900 / 1800; demo 15 / 30
}
```

12. Frontend: ecrane și componente

Aplicația are două rute: `/` (poziția controlorului, pe ecranul mare) și `/demo` (panoul de demo, pe telefon). Layout-ul principal e gândit pentru 1920×1080, pe proiector.

Layout-ul ecranului principal:

- **Bara de sus:** numele poziției („CRAIOVA APPROACH”), ceas UTC, stare conexiune, indicator „FRECVENȚĂ OCUPATĂ” când cineva vorbește.
- **Stânga, ~65% din lățime:** radarul.
- **Dreapta, sus:** panoul de comunicații (transcrierile ATC și pilot, cu instrucțiunea parsată sub fiecare).
- **Dreapta, mijloc:** stiva de alerte active.
- **Jos, pe toată lățimea:** fișele de zbor, câte una pe avion.
- **Peste radar, jos-centru:** bara PTT cu VU-metru.

Componentă	Ce afișează / face
`RadarScope.vue`	Canvas: inele de distanță la 10 NM, pista 09/27, poziția LRCV, simboluri avion cu urmă de 5 poziții, linie de direcție de 1 minut, etichetă (callsign / nivel × 100 și săgeată ↑↓ / viteză / LC mm:ss, timpul de la ultima comunicare, gri, galben la 15 min, roșu la 30 min). Click = selecție. Avion cu alertă = simbol roșu pulsat. Traiectoria din Clearance Probe = linie punctată.
`CommsPanel.vue`	Listă cronologică: vorbitor, callsign, transcriere, chip-uri cu instrucțiunea parsată, stare readback (✓ / ×), buton de redare audio
`AlertStack.vue`	Carduri de alertă (roșu/galben), cu tip, callsign, mesaj în clar și buton ACK; sunet la apariție
`FlightStrip.vue`	Fișa electronică: callsign, tip, nivel actual → autorizat, cap autorizat, viteză, apropiere, LAST CALL mm:ss; se colorează după starea conformității
`PttBar.vue`	Buton PTT mare + Space, VU-metru, stare: gata / înregistrez / transcriu / eroare
`DemoPanel.vue`	Panoul din secțiunea 11, folosit atât în `/demo`, cât și în sertarul lateral
`StatsOverlay.vue`	La final: erori injectate / detectate, timp mediu de detecție, latența STT

Store-uri Pinia: `useTrafficStore` (avioane, selecție), `useCommsStore` (transmisii, stare PTT), `useAlertStore` (alerte active și istoric), `useDemoStore` (`FaultConfig`, statistici). Un singur composable `useSocket()` conectează evenimentele la store-uri.

Audio: `useMicrophone()` pornește `MediaRecorder` la apăsarea Space și trimite blob-ul la eliberare (minimum 300 ms, ca să ignore apăsările accidentale). `useRadioVoice()` redă audio-ul pilotului prin Web Audio: filtru `bandpass` centrat la ~1500 Hz, distorsiune ușoară, zgomot alb la nivelul din panou și un „click” de squelch la început și la final.

13. Plan pe oameni și ore

Presupunere: 4 oameni, ~24 de ore. Ora 0–1 o faceți împreună: scrieți `packages/shared` (tipurile din secțiunile 7, 8 și 11) și porniți scheletul monorepo-ului. După asta fiecare lucrează pe mock-uri până la integrare.

Om	Rol	Orele 1–8	Orele 8–16	Orele 16–24

A	Voce și parser	`SttService`, normalizare, reguli, `phrases.json`	`CallsignResolver`, `LlmParser`, testele Jest	Înregistrări cu voci reale, măsurători, tuning
B	Simulator și siguranță	`SimulationService`, scenariul Craiova, gateway Socket.IO	`PilotService`, `ReadbackChecker`, `ConformanceMonitor`, `FaultService`	`ClearanceProbe`, `CommsWatchdog`, Postgres, statistici
C (tu)	Frontend principal	`RadarScope` pe date mock, layout, store-uri	`CommsPanel`, `AlertStack`, `FlightStrip`, `PttBar`, sunete	Polish vizual după designul din Claude Design
D	Demo, audio, pitch	`useRadioVoice`, `TtsService`, `DemoPanel`	Ruta `/demo` pe telefon, Docker (secțiunea 15), integrare cap-coadă	Video de backup, scriptul de pitch, repetiții

Puncte de control:

- [ ] Ora 8: o frază spusă la microfon ajunge ca instrucțiune pe radar
- [ ] Ora 12: pilotul răspunde vocal și avionul execută
- [ ] Ora 16: toate erorile din panou produc alerta așteptată (MVP gata)
- [ ] Ora 20: feature freeze, doar bug-uri și polish
- [ ] Ora 22: video de backup înregistrat
- [ ] Ora 23: minim 5 repetiții complete, inclusiv una cu un străin la microfon

14. Riscuri și backup-uri

Risc	Ce faceți
STT greșește callsign-uri și cifre	`prompt` cu vocabularul sesiunii, normalizare, potrivire fuzzy doar pe avioanele active, selecția de pe radar ca rezervă
WiFi-ul pică la pitch	Hotspot de pe telefon; ruta `/transmissions/text` ca rezervă; audio-ul pilotului pre-generat și salvat în cache pentru frazele din script; video de backup
Latență mare	Măsurați-o din prima zi; LLM-ul nu e pe calea principală; alertele nu așteaptă niciun API extern
Microfonul laptopului prinde sala	Microfon USB sau căști cu microfon; PTT, nu ascultare continuă
Browserul cere permisiune de microfon în timpul pitch-ului	Acordați-o dinainte; rulați pe `localhost` sau HTTPS, altfel `getUserMedia` e blocat
Alarme false în tranziție	Fereastră de persistență de 4 secunde, praguri din secțiunea 10
Întrebarea „ce se întâmplă dacă AI-ul greșește?”	Sistemul doar avertizează, nu dă instrucțiuni; controlorul decide; fiecare alertă arată transcrierea și motivul
Întrebarea „există deja?”	Există cercetare pe recunoașterea vocii în ATC; verificați înainte de pitch ce proiecte și rezultate puteți cita corect

15. Docker: pornire dintr-o comandă

Tot proiectul pornește cu `docker compose up --build`, pe orice laptop cu Docker Desktop, fără Node sau Postgres instalate. După ~2–3 minute la primul build, aplicația e la **http://localhost:8080**.

Pentru juriu, pașii sunt:

```
git clone <repo> && cd towerwhisper
cp .env.example .env        # pune cheile API (sau lasă gol pentru modul fără chei)
docker compose up --build
```

Serviciile: trei containere. Doar `web` expune un port; `api` și `db` sunt accesibile doar în rețeaua internă Docker.

Serviciu	Imagine	Rol
`web`	`nginx:1.27-alpine` cu build-ul Vite	Servește SPA-ul și face proxy pentru `/api` și `/socket.io` spre `api` (același origin, fără CORS)

`api`	`node:20-alpine` cu build-ul NestJS	Backend-ul; pornește doar după ce Postgres e sănătos
`db`	`postgres:16-alpine`	Date persistente într-un volum

```yaml
# docker-compose.yml
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: towerwhisper
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-towerwhisper}
      POSTGRES_DB: towerwhisper
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U towerwhisper"]
      interval: 3s
      retries: 20

  api:
    build: { context: ., dockerfile: apps/api/Dockerfile }
    env_file:
      - path: .env
        required: false   # pornește și fără .env (Compose 2.24+)
    environment:
      DATABASE_URL: postgres://towerwhisper:${POSTGRES_PASSWORD:-towerwhisper}@db:5432/towerwhisper
      PORT: 3000
    volumes:
      - audio:/app/data/audio
    depends_on:
      db: { condition: service_healthy }
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:3000/api/health"]
      interval: 3s
      retries: 30
    restart: unless-stopped

  web:
    build: { context: ., dockerfile: apps/web/Dockerfile }
    ports:
      - "8080:80"
    depends_on:
      api: { condition: service_healthy }
    restart: unless-stopped

volumes:
  pgdata:
  audio:
```

```dockerfile
# apps/api/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
RUN corepack enable
COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
COPY packages/shared/package.json packages/shared/
COPY apps/api/package.json apps/api/
RUN pnpm install --frozen-lockfile
COPY packages/shared packages/shared
COPY apps/api apps/api
RUN pnpm --filter shared build && pnpm --filter api build

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app /app
EXPOSE 3000
CMD ["node", "apps/api/dist/main.js"]
```

```dockerfile
# apps/web/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
RUN corepack enable
COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
COPY packages/shared/package.json packages/shared/
COPY apps/web/package.json apps/web/
RUN pnpm install --frozen-lockfile
COPY packages/shared packages/shared
COPY apps/web apps/web
RUN pnpm --filter shared build && pnpm --filter web build

FROM nginx:1.27-alpine
COPY apps/web/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/apps/web/dist /usr/share/nginx/html
```

```
# apps/web/nginx.conf
server {
  listen 80;
  root /usr/share/nginx/html;
  client_max_body_size 10m;             # audio de la PTT

  location /api/ {
```

```
    proxy_pass http://api:3000;
  }

  location /socket.io/ {
    proxy_pass http://api:3000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_read_timeout 3600s;
  }

  location / {
    try_files $uri $uri/ /index.html;    # rutele Vue: /, /demo
  }
}
```

```
# .env.example
OPENAI_API_KEY=          # Whisper + TTS
ANTHROPIC_API_KEY=       # parser de rezervă
POSTGRES_PASSWORD=towerwhisper
```

Ce trebuie să respecte codul ca să meargă în Docker:

- NestJS cu `app.setGlobalPrefix('api')`, un endpoint `GET /api/health` și `listen(3000, '0.0.0.0')`.
- Frontend-ul folosește doar căi relative (`/api`, `io('/live')`), deci nu are nevoie de variabile de build.
- TypeORM cu `synchronize: true` (acceptabil pentru hackathon) creează tabelele la pornire. Scenariul Craiova se încarcă din JSON, nu din bază.
- Un `.dockerignore` cu `node_modules`, `dist`, `.env`, ca build-ul să fie rapid și să nu intre cheile în imagine.

Modul fără chei. Dacă `.env` nu are chei, aplicația pornește oricum: PTT-ul e înlocuit de un câmp text cu frazele din script, iar vocea pilotului vine din fișiere audio pre-generate, incluse în repo (`apps/api/assets/tts-cache`). Juriul poate testa tot fluxul și toate erorile fără niciun cont.

Capcane:

- **Microfonul merge doar pe `localhost` sau HTTPS.** Ecranul controlorului se deschide la `http://localhost:8080` pe laptopul cu microfon. Telefonul cu panoul de demo poate intra pe `http://<ip-laptop>:8080/demo`, pentru că nu are nevoie de microfon.
- **Nu puneți chei reale în repo.** Dacă juriul rulează proiectul cu chei, dați-le o cheie temporară cu limită de cheltuieli și revocați-o după jurizare.
- **WiFi-ul de la hackathon poate fi lent la `docker pull`.** Faceți build-ul o dată dinainte, iar `docker compose up` ulterior pornește în câteva secunde, din cache.

☐ Test pe un laptop „curat” (al cuiva care nu a lucrat la proiect): clone → `.env` → `docker compose up --build` → demo complet

16. Prompt pentru Claude Design

Copiați blocul de mai jos în Claude Design. E în engleză pentru că toată interfața e în engleză ATC.

```
Design the UI for TowerWhisper, a web app that acts as an AI co-pilot for an air
traffic controller. The controller speaks into a microphone (push-to-talk). The app
transcribes the instruction, parses it into structured data, listens to the simulated
pilot's readback, and raises alerts when the readback is wrong (hearback mismatch) or
when the aircraft does not do what it was told (wrong turn, level bust). It will be
shown live on a projector at a hackathon, and one judge will speak into the
microphone during the demo.

DELIVERABLES
1. Main controller screen, desktop 1920x1080 (projector).
2. Demo control panel, mobile 390x844 (operated secretly on a phone during the pitch).
3. End-of-demo stats overlay on top of the main screen.
Show the main screen in these states: (a) normal traffic, (b) controller holding PTT
and talking, (c) transcribing / waiting for readback, (d) red HEARBACK MISMATCH alert,
(e) amber READBACK INCOMPLETE alert next to a red WRONG TURN alert on another aircraft, (f) NO CONTACT: one aircraft amber
at 16 min without communication, another red at 31 min.

MAIN SCREEN LAYOUT
- Top bar (56px): position name "CRAIOVA APPROACH", UTC clock, connection status dot,
  a "FREQUENCY BUSY" indicator that lights up while anyone is transmitting, session timer.
- Left, about 65% width: radar scope. Dark background, range rings every 10 NM with
  small labels, runway 09/27 drawn at the airport LRCV in the centre, a few waypoint
  triangles. 6-8 aircraft: a small symbol, 5 fading history dots behind it, a 1-minute
  heading vector line, and a 3-line data block: line 1 callsign (ROT301), line 2 current
  level / cleared level with a climb or descend arrow (030 v 030), line 3 ground speed
  and type (180 AT76), line 4 "LC 08:35" = time since last contact, grey normally, amber
  from 15 min, red from 30 min; an aircraft past a limit turns amber or red on the radar.
  Selected aircraft has a highlight ring. Aircraft in alert: symbol
  and data block turn red and pulse. Optional dashed predicted path line for one aircraft.
- Right column, top: Communications panel, a chronological feed. Each entry: speaker tag
  (ATC / PILOT), callsign, timestamp, transcript text with key values emphasised, and a
  row of parsed-instruction chips below it (ALT 3000 v, HDG 120 L, ILS 09). Pilot entries
  show a readback status icon: check (match), cross (mismatch), half (incomplete), plus
```

```
  a small play button for the audio.
- Right column, middle: Alert stack. Cards with severity colour bar (red = warning,
  amber = caution), alert type in caps (HEARBACK MISMATCH), callsign, a plain-English
  reason ("ROT301 cleared 3000 ft, read back 5000 ft"), seconds since raised, and an
  ACK button. Newest on top. Max 3 visible.
- Bottom strip, full width: electronic flight strips, one per aircraft, horizontally
  scrollable. Each strip: callsign, aircraft type, current level -> cleared level,
  cleared heading, speed, approach. Strip edge colour reflects conformance
  (neutral / amber / red). Each strip also shows LAST CALL mm:ss, highlighted amber
  from 15 min and red from 30 min without communication.
- Floating over the bottom centre of the radar: PTT bar. Large rounded push-to-talk
  button with "HOLD SPACE TO TALK", a live VU meter, and a status label: READY /
  TRANSMITTING (red, active glow) / TRANSCRIBING (spinner) / ERROR.

DEMO PANEL (mobile)
- Header: "DEMO CONTROL", connection status, live counter
  "Injected 4 / Detected 4 / avg 1.8 s".
- Target selector: Selected aircraft / pick callsign / Any.
- Mode toggle: ONCE (default) / PERSISTENT.
- Section "Readback faults" as large tappable tiles, each with a short label and the
  expected alert as a small tag: Wrong altitude, Wrong heading, Wrong turn direction,
  Wrong runway, Wrong callsign, Incomplete readback, No response.
- Section "Execution faults": Turn wrong way, Level bust, Wrong vertical,
  Ignore instruction.
- Section "Comms timer": limits toggle (15 / 30 min real, 15 / 30 s demo), tiles
  Age to 15 min (NO CONTACT amber) and Age to 30 min (NO CONTACT red), Reset timer on target.
- Section "Scenario": Spawn conflict, Stepped-on transmission, Radio noise slider
  0-100%, time speed segmented control (Pause / 1x / 2x / 4x), Reset scenario
  (destructive style).
- An armed fault tile shows a clear armed state until it fires, then flashes once and
  returns to idle.

STATS OVERLAY
Centered modal over a dimmed main screen: faults injected, faults detected, mean
detection time, parser accuracy on the test set, mean speech-to-text latency. Big
numbers, short labels.

VISUAL DIRECTION
- Modern ATC console, not a sci-fi movie screen: calm, dense, highly legible from
  5 metres away on a projector.
- Dark theme only. Deep blue-grey background, slightly lighter panels, thin 1px
  borders, no gradients, no glassmorphism, no glow except the active PTT state and
  pulsing alerts.
- Colour carries meaning only: neutral cool white/grey for normal traffic, cyan for the
  selected aircraft and controller (ATC) entries, a second calm hue for pilot entries,
  amber for caution, red for warning, green only for a confirmed readback check.
  No decorative colour.
- Typography: a monospaced font for all data (callsigns, levels, headings, transcripts'
  key values), e.g. JetBrains Mono or IBM Plex Mono; a clean sans (Inter) for labels
  and UI chrome. Data block text at least 14px at 1080p; alert reason text at least 16px.
- Sample data: ROT301 (TAROM 301, AT76), WZZ44 (WIZZ 44, A21N), ROT612 (TAROM 612, B738),
  LOT2RP (LOT, E195), RYR8KQ (RYANAIR, B38M), DLH9TC (LUFTHANSA, CRJ9). Sample transcript:
  "TAROM 301, descend altitude 3000 feet, turn left heading 120, cleared ILS approach
  runway 09." Sample bad readback: "Descend 5000, left heading 120, cleared ILS 09,
  TAROM 301."

CONSTRAINTS
- Will be implemented in Vue 3 with plain CSS: define colours, spacing and type as CSS
  custom properties (design tokens) and keep components reusable: RadarScope, CommsPanel,
  CommsEntry, InstructionChip, AlertCard, FlightStrip, PttBar, DemoTile, StatsOverlay.
- The radar will be drawn on a canvas, so specify its visual rules (sizes, colours,
  line weights, label offsets) rather than intricate illustration.
- WCAG AA contrast for all text; alerts must not rely on colour alone (icon + text label).
- No real airline logos or branding; callsigns as plain text only.
```